

วันที่ 5

Routing & Packing

ปัญหาโลจิสติกส์

TSP · Vehicle Routing · Bin Packing · Knapsack



TSP/VRP

หาเส้นทางสั้นสุด



Bin Packing

minimize กล่อง



Knapsack

เลือก maximize ทำไร

กำหนดการวันที่ 5

09:00–09:45

**TSP — Travelling Salesman***circuit() global constraint — Hamiltonian path*

09:45–11:00

**Vehicle Routing Problem***หลาย vehicle, capacity, depot copies, minimize ระยะทาง*

11:00–12:00

**Workshop 5A***VRP รถส่งสินค้า — 8 ลูกค้า, 3 รถ, capacity 100 กก.*

13:00–14:30

**Bin Packing***bin_packing_capa() + symmetry breaking + ปัญหาจริง*

14:30–15:30

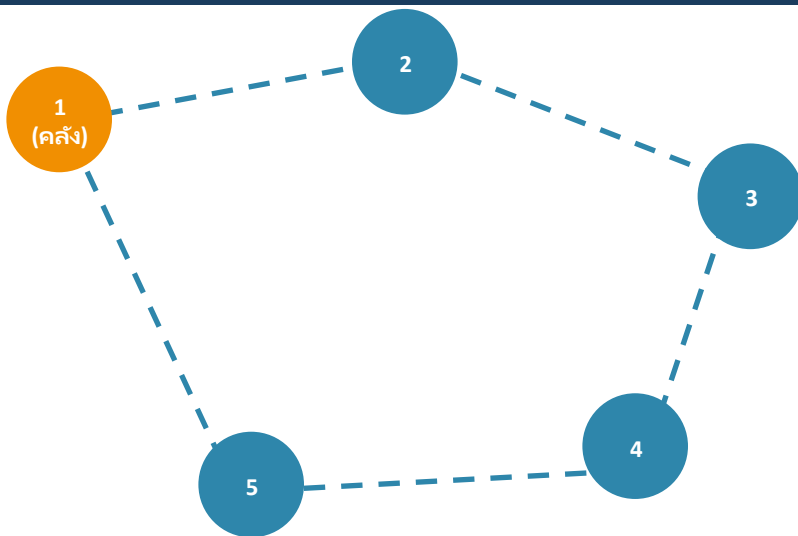
**Knapsack & Set Variables***0/1 Knapsack, var set of ITEMS, multiple knapsack*

15:30–16:00

**Workshop 5B***Bin Packing คอนเทนเนอร์ 12 รายการ + สินค้าอันตราย*

TSP — Travelling Salesman Problem

เยี่ยมชมทุกจุดครั้งเดียว กลับจุดเริ่มต้น → Minimize ระยะทางรวม



เส้นทาง: 1→2→3→4→5→1

ตัวแปรหลัก:

```
% next[i] = เมืองถัดไป  
array[CITIES] of  
  var CITIES: next;  
  
% circuit: บังคับ  
% Hamiltonian cycle  
constraint  
  circuit(next);
```



circuit() = 1 constraint แทนที่ constraint ซับซ้อน หาเส้นทาง Hamiltonian

circuit() + TSP Model

```
include "circuit.mzn";

int: n = 5;
set of int: CITIES = 1..n;
array[CITIES,CITIES] of int: dist;

% เมืองถัดไปจากแต่ละเมือง
array[CITIES] of var CITIES: next;

% Hamiltonian circuit
constraint circuit(next);
% ห้ามอยู่กับที่
constraint forall(i in CITIES)(next[i] != i);

solve minimize
  sum(i in CITIES)(dist[i, next[i]]);
```

◀ ตัวแปรหนึ่งตัว
ต่อหนึ่งเมือง

◀ circuit() บังคับว่า
ทุกเมืองถูกเยี่ยมชม 1 ครั้ง

◀ objective: รวมระยะทาง
ตาม path ที่ next[] กำหนด

next = [2, 3, 4, 5, 1] → เส้นทาง: 1→2→3→4→5→1 ระยะทาง: 41 กม.

VRP — Vehicle Routing Problem

TSP + หลาย vehicle + capacity จำกัด → แต่ละรถต้องออกและกลับ depot

เทคนิค: Depot Copies

สร้าง node พิเศษสำหรับแต่ละรถ (depot copy): $n_nodes = n_customers + n_vehicles$

node 1.. $n_customers$ = ลูกค้า | node $n_customers+1..n_nodes$ = depot แต่ละคัน

circuit() บน NODES ทั้งหมด → แต่ละรถเริ่มจาก depot copy → เยี่ยมลูกค้า → กลับ depot copy อื่น

Capacity Constraint:

```
% load[i] = demand ของ i + load ที่สะสมมาจาก next[i]
constraint forall(i in CUSTOMERS)(
  load[i] = demand[i] + (if next[i] in DEPOT then 0 else load[next[i]] endif));
```

VRP — โมเดล MiniZinc สมบูรณ์

```
include "circuit.mzn";

int: n_cust; int: n_veh; int: capacity;
int: n_nodes = n_cust + n_veh;
set of int: CUST=1..n_cust; DEPOT=n_cust+1..n_nodes;
array[CUST] of int: demand;
array[1..n_nodes,1..n_nodes] of int: dist;

array[1..n_nodes] of var 1..n_nodes: next;
array[CUST] of var 0..capacity: load;

constraint circuit(next);
constraint forall(i in CUST)(load[i]=demand[i]+(
    if next[i] in DEPOT then 0 else load[next[i]] endif));

constraint forall(i in CUST)(load[i] <= capacity);

solve minimize sum(i in 1..n_nodes)(dist[i,next[i]]);
```



Workshop 5A — VRP รถส่งสินค้า

เวลา: 60 นาที | ไฟล์: workshop5a_starter.mzn + workshop5a_data.dzn

โจทย์: คลัง 1 แห่ง, ลูกค้า 8 ราย, รถ 3 คัน capacity 100 กก.

Demand: [25,30,15,40,20,35,10,45] กก.

1

เพิ่ม circuit constraint:

2

เพิ่ม Capacity constraint:

3

เพิ่ม load $\leq$ capacity:

4

solve minimize ระยะทาง:

5

ตีความผล:

`next[depot_v]` บอกลูกค้าแรกของรถ `v` — ติดตาม `next[]` จนกลับ depot

ช่วงบ่าย

Bin Packing และ Knapsack

- bin_packing_capa() global constraint

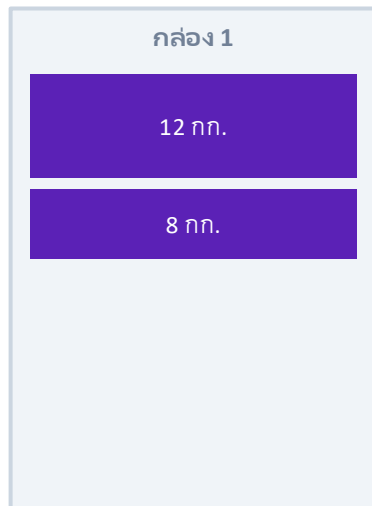
- Symmetry breaking — ลด solution space

- var set of ITEMS — knapsack แบบ set variable

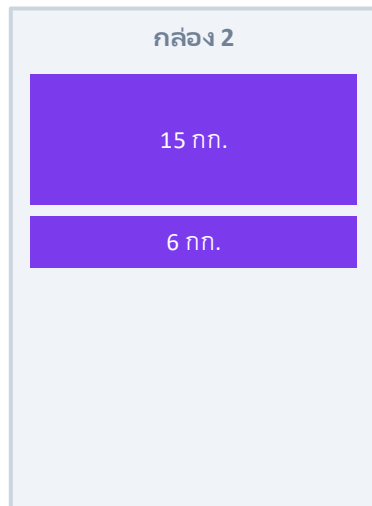
- Multiple knapsack + additional constraints

Bin Packing — จัดของลงกล่อง

Minimize จำนวนกล่องที่ใช้ | แต่ละกล่องต้องไม่เกิน capacity



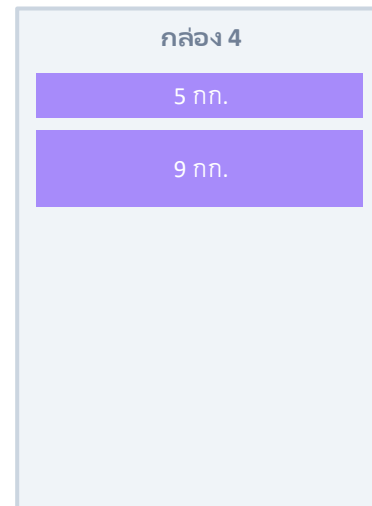
รวม: 20 กก.



รวม: 21 กก.



รวม: 28 กก.



รวม: 14 กก.

ตัวแปรหลัก: $\text{bin_of}[i]$ = กล่องที่ item i อยู่

Bin Packing — โมเดล MiniZinc

```
include "bin_packing_capa.mzn";

int: n_items, bin_capacity; array[1..n_items] of int: item_size;
set of int: ITEMS=1..n_items; BINS=1..n_items;
array[ITEMS] of var BINS: bin_of;

% bin_packing_capa: ทุกกล่องไม่เกิน capacity
constraint bin_packing_capa(bin_capacity, bin_of, item_size);

% Symmetry: bin_of[i] <= bin_of[i+1]+1
constraint forall(i in 1..n_items-1)(bin_of[i]<=bin_of[i+1]+1);

var int: n_used=sum(b)(exists(i in ITEMS)(bin_of[i]=b));

solve minimize n_used;
```

◀ bin_packing_capa:
specialised global constraint

◀ Symmetry breaking:
กล่อง 1,2,3 แทนที่ 1,3,2

◀ n_used = จำนวนกล่อง
ที่มีของอย่างน้อย 1 ชิ้น

Symmetry Breaking — ทำไมสำคัญ?

Symmetry: คำตอบที่ 'เหมือนกัน' เพียงแต่เปลี่ยนชื่อกล่อง → Solver เสียเวลาหาซ้ำ

ตัวอย่าง: 3 ชิ้น → 2 กล่อง (คำตอบเดียวกัน 3 แบบ):

✓ เก็บแค่นี้

`bin_of=[1,1,2]`

A,B ใน กล่อง 1, C ใน กล่อง 2

✗ เหมือนกัน

`bin_of=[1,2,1]`

A,C ใน กล่อง 1, B ใน กล่อง 2

✗ เหมือนกัน

`bin_of=[2,2,1]`

A,B ใน กล่อง 2, C ใน กล่อง 1

วิธีแก้: `bin_of[i] <= bin_of[i+1] + 1`

% บังคับให้ item ที่ i มี bin number <= item i+1 + 1

% ผลลัพธ์: bin 1 เต็มก่อน → bin 2 → bin 3 (ไม่สลับกัน)

`constraint forall(i in 1..n_items-1)(bin_of[i] <= bin_of[i+1] + 1);`



Symmetry breaking อาจลด solve time ได้มากถึง 10-100x สำหรับปัญหาใหญ่

Knapsack — เลือก Maximize กำไร

เลือกสิ่งของที่จะหิ้ว | $\text{weight} \leq \text{capacity}$ | Maximize profit

2 วิธีใน MiniZinc:

วิธีที่ 1: Boolean Array

```
array[ITEMS] of var 0..1: sel;  
constraint sum(i)(cost[i]*sel[i])<=budget;  
solve maximize sum(i)(profit[i]*sel[i]);  
  
% sel[i]=1 → เลือก, 0 → ไม่เลือก
```

วิธีที่ 2: Set Variable

```
var set of ITEMS: chosen;  
constraint sum(i in chosen)(cost[i])<=budget;  
solve maximize sum(i in chosen)(profit[i]);  
  
% chosen = {P1,P3,P5} อ่านง่ายกว่า
```



Workshop 5B — Bin Packing คอนเทนเนอร์

เวลา: 30 นาที | ไฟล์: workshop5b_starter.mzn

โจทย์: สินค้า 12 รายการ [15,22,8,30,11,25,18,7,33,14,20,9] กก.
คอนเทนเนอร์ 50 กก./ใบ ราคา 3,000 บาท/ใบ — Minimize จำนวนใบ

- | | | |
|---|--------------------------|--|
| 1 | เพิ่ม bin_packing_capa: | |
| 2 | เพิ่ม symmetry breaking: | |
| 3 | solve minimize n_used: | |
| 4 | ท้าทาย: สินค้าอันตราย: | สินค้า 1,3,5 ห้ามอยู่ตู้เดียวกัน |
| 5 | คำนวณ ROI: | ตู้ 80 กก. ราคา 4,500 บาท — ประหยัดได้กี่ตู้? คุ่มไหม? |

สรุปวันที่ 5 — สิ่งที่ต้องจำ



circuit() — หัวใจของ TSP และ VRP

constraint circuit(next) → Hamiltonian cycle | objective: sum(dist[i,next[i]])



VRP — Depot Copies + Load Tracking

n_nodes = n_cust + n_veh | load[i] = demand[i] + (if next[i] in DEPOT then 0 else load[next[i]])



Bin Packing — bin_packing_capa + symmetry breaking

bin_packing_capa(cap, bin_of, sizes) | bin_of[i] <= bin_of[i+1]+1 ลด duplicate solutions



Knapsack — boolean array หรือ set variable

sum(i)(cost[i]*sel[i]) <= budget | var set of ITEMS: chosen (อ่านง่ายกว่า)

Quick Reference: Routing Patterns

```
% — TSP —————  
  
include "circuit.mzn";  
array[CITIES] of var CITIES: next;  
constraint circuit(next); constraint forall(i)(next[i]!=i);  
solve minimize sum(i in CITIES)(dist[i,next[i]]);
```

```
% — VRP (depot copies) —————  
  
int: n_nodes = n_cust + n_veh;  
set of int: DEPOT = n_cust+1..n_nodes;  
array[1..n_nodes] of var 1..n_nodes: next;  
array[CUST] of var 0..cap: load;  
  
constraint circuit(next);  
constraint forall(i in CUST)(load[i]=demand[i]+(  
    if next[i] in DEPOT then 0 else load[next[i]] endif));  
  
constraint forall(i in CUST)(load[i] <= cap);
```

Quick Reference: Packing Patterns

```
% — BIN PACKING —
include "bin_packing_capa.mzn";
array[ITEMS] of var BINS: bin_of;
constraint bin_packing_capa(bin_capacity, bin_of, item_size);
% Symmetry breaking:
constraint forall(i in 1..n-1)(bin_of[i] <= bin_of[i+1]+1);
% Item ห้ามอยู่กล่องเดียวกัน:
constraint forall(i,j in DANGER where i<j)(bin_of[i]!=bin_of[j]);

% — KNAPSACK —
% วิธีที่ 1 - boolean
array[ITEMS] of var 0..1: sel;
constraint sum(i)(weight[i]*sel[i]) <= capacity;
solve maximize sum(i)(profit[i]*sel[i]);

% วิธีที่ 2 - set variable
var set of ITEMS: chosen;
constraint sum(i in chosen)(weight[i]) <= capacity;
```


ข้อผิดพลาดที่พบบ่อย — วันที่ 5

✗ `next[i] != i` % ลืม constraint นี้

✓ `constraint forall(i in CITIES)(next[i] != i);`

circuit() อนุญาต self-loop — ต้องห้ามเองสำหรับ TSP

✗ `dist[DEPOT_i, DEPOT_j] = 0` % depot ถึง depot

✓ ระยะทาง depot→depot ควรเป็น 0 (รถที่ไม่ออก)

VRP: depot copies ควรมีระยะทาง 0 ถึงกัน เพื่อให้รถที่ไม่ต้องออกข้ามกันได้

✗ `bin_of[1] = 1` % symmetry breaking ซ้ำเกิน

✓ `forall(i in 1..n-1)(bin_of[i] <= bin_of[i+1]+1);`

Symmetry breaking แบบ `bin_of[i] <= bin_of[i+1]+1` ถูกต้อง — ไม่ตัด optimal solution

✗ `var set of ITEMS: chosen;`

✓ `constraint sum(i in chosen)(weight[i]) <= cap;`

set variable ต้องใช้ constraint และ objective เหมือน variable ทั่วไป — ไม่ใช่กำหนดค่าตรงๆ

Routing & Packing ในอุตสาหกรรมจริง



Logistics / Supply Chain

- ▶ VRP: รถส่งพัสดุ/ร้านค้า
- ▶ Multi-depot VRP: หลายคลัง
- ▶ Time-window VRP: ต้องส่งในช่วงเวลาที่กำหนด



Aviation / Transportation

- ▶ TSP: เส้นทางบิน, ขนส่งสาธารณะ
- ▶ Fleet routing: กำหนดรถโดยสาร
- ▶ Crew scheduling: จัดลูกเรือ



Warehouse / Manufacturing

- ▶ Bin Packing: จัดสินค้าขึ้นรถ
- ▶ Container loading: ลงตู้เรือ
- ▶ Cutting stock: ตัดวัสดุ waste น้อย



Finance / IT

- ▶ Knapsack: เลือก project ลงทุน
- ▶ Resource allocation: จัด server
- ▶ Portfolio optimization

Day 6 Preview

Effective Modelling + Capstone Project

Variable bounds — ช่วย solver ตัด search space

Redundant constraints — เพิ่ม pruning

Symmetry breaking — รู้จักแล้ว วันนี้ฝึกใช้จริง

Debugging — UNSAT, slow solver, wrong results

Capstone: นำโมเดลไปใช้กับปัญหาจริงขององค์กร

เลือก Model ให้ถูกกับปัญหา

ปัญหา	ใช้	Key variable	Constraint
หาเส้นทางสั้นสุด (1 รถ)	TSP	next[CITIES]	circuit()
หลาย vehicle + capacity	VRP	next[NODES] + load	circuit() + cap
จัดสิ่งของลงกล่อง	Bin Packing	bin_of[ITEMS]	bin_packing_capa()
เลือก item maximize profit	Knapsack	sel[ITEMS] 0/1	sum(weight*sel)<=cap
เลือก item (อ่านง่าย)	Knapsack (set)	var set of ITEMS	sum in chosen <= cap
หลายกล่องต่างความจุ	Multi Knapsack	bin_of + capacity[b]	forall(b)sum<=cap[b]

Bonus: 2D Packing และ Cutting Stock

ขยายจาก 1D Bin Packing → 2D: จัด rectangle บนพื้นที่ 2 มิติ

```
% 2D Bin Packing: จัด rectangle (w[i] x h[i]) บน sheet W x H
```

```
array[ITEMS] of var 0..W: px; % ตำแหน่ง x
```

```
array[ITEMS] of var 0..H: py; % ตำแหน่ง y
```

```
% ไม่ overlap กัน (ต้องห่างอย่างน้อย 1 มิติ)
```

```
constraint forall(i,j in ITEMS where i<j)(
  px[i]+w[i]<=px[j] \/ px[j]+w[j]<=px[i] \/
  py[i]+h[i]<=py[j] \/ py[j]+h[j]<=py[i]);
```

ประยุกต์จริง:

Pallet loading

จัด box บนพาเลท ไม่ซ้อนเกิน height limit

Steel/paper cutting

ตัดแผ่นเหล็ก/กระดาษให้ waste น้อยสุด

PCB layout

จัด component บน circuit board